

Step 4 — A Reasoner Agent That Only Answers From What the Retriever Found — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — "use the context," said once, plainly

```
# agents/reasoner_agent.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.messages import AIMessage
from langchain_openai import ChatOpenAI

def reason_node(state):
    context = "\n--\n".join(c.page_content for c in state["found_chunks"])
    prompt = ChatPromptTemplate.from_messages([
        ("system", "Answer using only the context below."),
        ("human", "Context:\n{context}\n\nQuestion: {task}"),
    ])
    model = ChatOpenAI(model="gpt-4o-mini")
    chain = prompt | model | StrOutputParser()
    answer = chain.invoke({"context": context, "task": state["task"]})
    return {"messages": [AIMessage(content=answer)]}
```

Expected output, run on a question fully covered by the found chunks:

Damaged items can be returned within 30 days of delivery for a full refund.

Expected output, run on a question only *partially* covered (found chunks mention the 30-day window but say nothing about international orders, and the actual question asks about international returns specifically):

International orders can be returned within 30 days for a full refund.

This second answer looks fully grounded but isn't — "30 days for a full refund" is real, from the chunks; "international orders" specifically was never actually confirmed by anything retrieved, it's the model quietly extending the general policy to a case the context never covered. "Answer using only the context below" doesn't stop this blending — the model believes it *is* using the context, just filling a small, reasonable-seeming gap.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — an explicit "say so if it's missing" instruction

```
# agents/reasoner_agent.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.messages import AIMessage
from langchain_openai import ChatOpenAI

REASONER_SYSTEM_PROMPT = """Answer the user's question using ONLY the context
provided below. If the context does not contain the answer, say plainly that
you don't have enough information – do not use anything you already know
beyond what's in the context."""

def reason_node(state):
    context = "\n---\n".join(c.page_content for c in state["found_chunks"])
    prompt = ChatPromptTemplate.from_messages([
        ("system", REASONER_SYSTEM_PROMPT),
        ("human", "Context:\n{context}\n\nQuestion: {task}"),
    ])
    model = ChatOpenAI(model="gpt-4o-mini")
    chain = prompt | model | StrOutputParser()
    answer = chain.invoke({"context": context, "task": state["task"]})
    return {"messages": [AIMessage(content=answer)]}
```

```
# graph.py (excerpt – Step 3's placeholder replaced)
builder.add_node("reason", reason_node)
builder.add_conditional_edges("retrieve", route_after_retrieve, {"answer": "reason"})
builder.add_edge("reason", END)
```

Expected output, fully-covered question:

Damaged items can be returned within 30 days of delivery for a full refund.

Expected output, the same partial-coverage international-returns question:

```
International orders can be returned within 30 days for a full refund. <-- stil
```

Difference from Basic: "if the context does not contain the answer, say plainly that you don't have enough information" is a real improvement over Basic's bare "use only the context" — it now handles the case where the context is *entirely* silent on a topic. But it still doesn't stop the *partial*-coverage case above, because from the model's perspective the context isn't silent, it's just slightly under-specific, and "say you don't have enough information" doesn't clearly apply to "I have some information, I'm extending it a little."

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — an instruction that specifically targets partial blending

```
# agents/reasoner_agent.py
REASONER_SYSTEM_PROMPT = """Answer the user's question using ONLY the context
provided below. If the context does not contain the answer, say plainly that
you don't have enough information.
```

```
If any PART of your answer is not directly supported by the context, leave
that part out rather than filling it in from what you already know. It is
better to give a partial, clearly-scoped answer than a complete-sounding one
that extends beyond what the context actually confirms."""
```

Expected output, the same partial-coverage international-returns question:

```
The context confirms damaged items can be returned within 30 days for a full refu
but it doesn't specifically address international orders – I don't have enough
information to confirm whether that same policy applies internationally.
```

Same underlying model, same retrieved chunks — the only change is naming the exact failure mode ("leave that part out") instead of the more general "use only the context," which the model was already technically complying with in the blended answer above.

Approach 2 — a test built specifically to catch blending

```
# test_project3.py
from graph import build_graph

def test_reasoner_does_not_blend_memory_with_partial_context():
    graph = build_graph()
    # found_chunks deliberately cover the general policy but not the specific
    # international-orders case the question asks about
    result = graph.invoke({
        "task": "Does the 30-day return policy apply to international orders?",
        "messages": [],
        "found_chunks": [
            Document(page_content="Damaged items can be returned within 30 days o
```

```

    ],
    "grounded": True,
  })
  answer = result["messages"][-1].content.lower()
  assert "international" in answer
  assert "don't have enough information" in answer or "doesn't specifically add
  assert "yes" not in answer.split(".")[0].lower() # the first sentence should

```

Expected output:

```
test_reasoner_does_not_blend_memory_with_partial_context PASSED
```

This test constructs `found_chunks` directly rather than going through a real Retriever call — that's deliberate, it isolates the Reasoner's behavior from Step 3's search quality entirely, so a failure here can only mean the Reasoner's own instruction-following broke, not that the knowledge base changed.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's instruction handles context that's completely silent on a topic, but not context that's a *partial* match — the exact case that produces answers looking fully grounded while actually being part-invented. Approach 1 closes that gap with language that names blending specifically. Approach 2 is the proof: a test that hand-builds a partial-coverage scenario and checks the Reasoner actually flags the gap instead of quietly filling it — the kind of test the README's "check this by hand against a couple of test cases, not just 'it ran without error'" instruction is really asking for, made permanent and automatic.

Which one should you actually write? Both. The sharpened prompt is a few extra sentences that meaningfully change behavior on exactly the case that's hardest to catch by eye — a partially-blended answer reads just as fluently as a fully-grounded one, so catching it "by eye" in a quick manual test is genuinely unreliable without a rubric to check against. Approach 2's test is what actually operationalizes that rubric, and it's cheap to keep since it never needs a real vector store to run.